МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ТАРАСА ШЕВЧЕНКА

ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ ТА ТЕХНОЛОГІЙ

**Практична робота № 4**

з дисципліни **«Технології створення програмних продуктів»**

Тема: «Проектування системи»

**Виконав:**

студент гр. ІР-31

Петро Кравченко

**Перевірив:**

Зараховано від

Кравченко Ольга Віталіївна

_____

(підпис викладача)

Київ-2026

**ЗМІСТ**

1. МЕТА ТА ЗАВДАННЯ РОБОТИ ..... 3
2. ПРОЕКТУВАННЯ СИСТЕМИ ..... 5
2.1. Моделювання області застосування в нотації BPMN ..... 5
2.2. Функціональна модель системи в нотації IDEF0 ..... 9
2.3. Концептуальна модель системи в нотації UML ..... 14
2.4. Логічна модель системи ..... 22
2.5. Фізична модель системи ..... 30
2.6. Реалізація моделей у CASE-засобах ..... 36
ВИСНОВКИ ..... 41
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ ..... 42

# 1. МЕТА ТА ЗАВДАННЯ РОБОТИ

**Мета роботи:** виконати проектування програмної системи засобами нотацій BPMN, IDEF0 та UML: описати область застосування, бізнес-процеси й функції системи, а також структуру та взаємозв’язок її вузлів і модулів на концептуальному, логічному та фізичному рівнях.

**Завдання роботи** [1]: підготувати розділ «Проектування системи» проектної роботи з діаграмами BPMN, UML та IDEF, що описують область застосування системи та взаємозв’язок вузлів і модулів; кожен рисунок детально описати технічною мовою від третьої особи. Структуру розділу побудовано за зразком опису діаграм [2]: від концептуальної моделі предметної області до логічної та фізичної моделей. Поєднання нотацій UML, BPMN та IDEF0 для проектування IoT-систем застосовують і в роботах кафедри [3].

Об’єкт проектування — IoT-платформа «Щит-Лінк» на тему «Розподілена IoT-система цілевказівки та координації дронів для перехоплення повітряних цілей». Вимоги до системи визначено в практичній роботі № 1 [4], архітектуру IoT-рішення — у практичній роботі № 2 [5], склад обладнання — у практичній роботі № 3 [6]. Звіт оформлено відповідно до ДСТУ 3008:2015 [7].

*Примітка: РЦПДО «Щит-Північ» і IoT-платформа «Щит-Лінк» є навчальними умовними моделями; усі назви, процеси та показники наведено суто з навчально-ілюстративною метою.*

Склад моделей розділу наведено в табл. 4.1. Відповідно до технології RUP діаграми прецедентів визначають функції системи, діаграми діяльності — сценарії роботи користувача, діаграми класів — модель даних, діаграми послідовності й кооперації — модель проекту, діаграми компонентів і розгортання — моделі реалізації та розміщення [8].

Таблиця 4.1 — Склад моделей розділу «Проектування системи»

| Рівень | Нотація, діаграма | Що моделює | Рисунки |
|---|---|---|---|
| Область застосування | BPMN: діаграми процесів | Бізнес-процеси центру з участю системи | 4.1, 4.2 |
| Функції системи | IDEF0: контекстна діаграма та декомпозиція | Функції, потоки даних і вузли-механізми | 4.3, 4.4 |
| Концептуальна модель | UML: прецедентів, станів, класів | Призначення системи, життєвий цикл місії, словник предметної області | 4.5–4.7 |
| Логічна модель | UML: діяльності, послідовності, кооперації | Алгоритм і взаємодію об’єктів у сценарії призначення розрахунку | 4.8–4.10 |
| Фізична модель | UML: класів, компонентів, розгортання | Схему даних, модулі та вузли системи | 4.11–4.13 |
| Реалізація моделей | CASE-засоби, генерація SQL DDL | Засоби побудови моделей, обробку моделей і генерацію схеми БД | 4.14, 4.15 |

# 2. ПРОЕКТУВАННЯ СИСТЕМИ

## 2.1. Моделювання області застосування в нотації BPMN

Нотація BPMN (Business Process Model and Notation) описує бізнес-процес як послідовність подій, дій і логічних операторів (шлюзів), розподілених за учасниками. Пул відображає учасника процесу, доріжки всередині пулу — ролі або підсистеми; суцільні стрілки позначають потік управління всередині пулу, пунктирні — потоки повідомлень між пулами. Нижче наведено дві моделі «як буде» (TO-BE), у яких IoT-платформа «Щит-Лінк» виконує частину дій автоматично, а рішення залишаються за людьми.

### *2.1.1. Процес формування повітряної обстановки*

Процес формування повітряної обстановки (рис. 4.1) розпочинається стартовою подією-повідомленням «Виявлення надійшло»: згорнутий пул «Сенсорна мережа» надсилає повідомлення «Виявлення». Пул центру має три доріжки: «Периферійний шлюз», «IoT-ядро» та «Аналітик обстановки». Сервісне завдання шлюзу «Перевірити джерело й нормалізувати» автентифікує сенсор за сертифікатом і перетворює дані в канонічну модель. Виключний шлюз «Дані коректні?» відкидає некоректні повідомлення (кінцева подія «Відхилено»), а коректні передає до IoT-ядра.

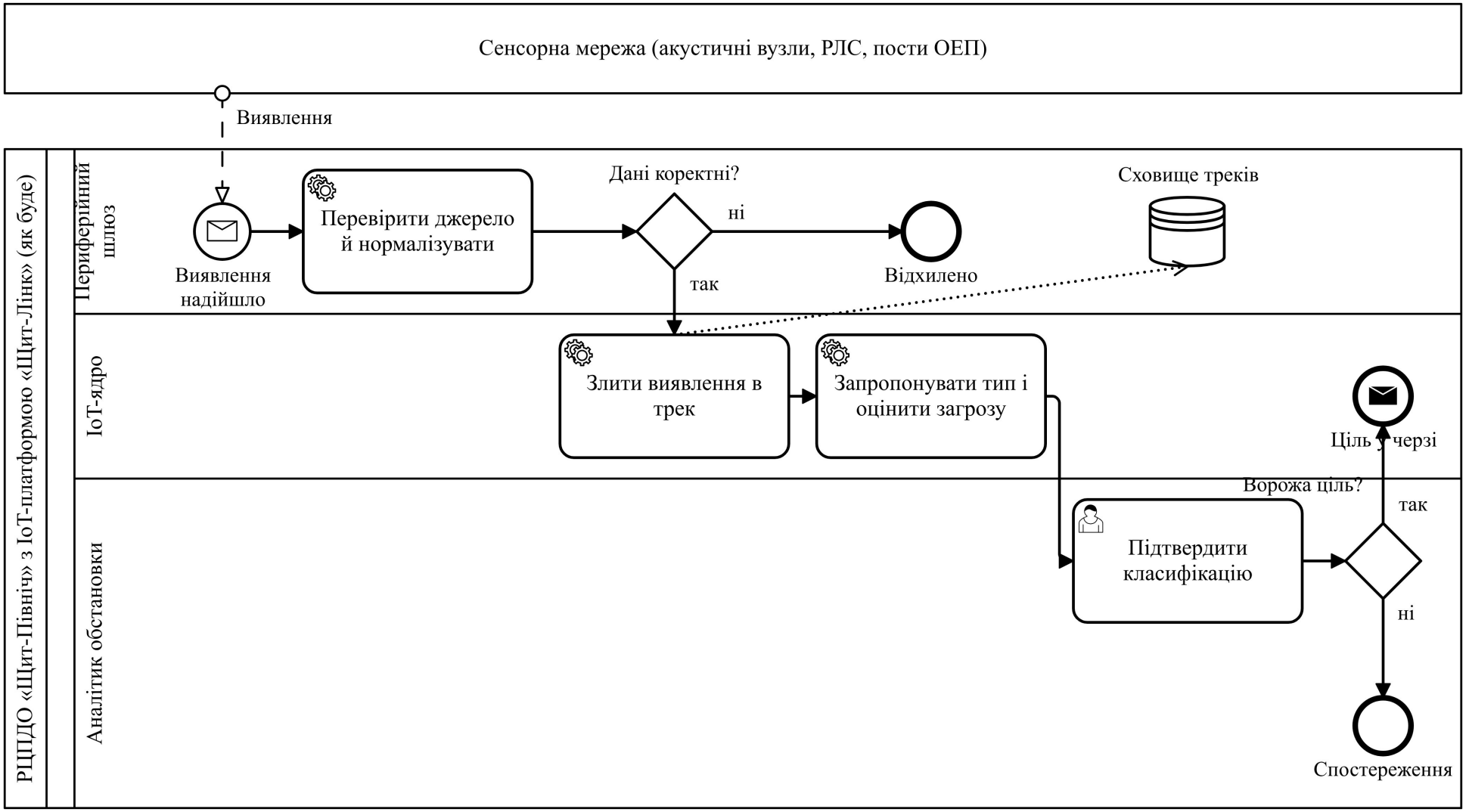



Рис. 4.1 — BPMN-модель процесу формування повітряної обстановки

У доріжці «IoT-ядро» сервісне завдання «Злити виявлення в трек» оновлює повітряний трек і записує його до сховища треків (об’єкт даних), після чого завдання «Запропонувати тип і оцінити загрозу» формує підказку класифікатора та оцінку загрози. Рішення про належність цілі ухвалює людина: користувацьке завдання «Підтвердити класифікацію» в доріжці аналітика завершується шлюзом «Ворожа ціль?». Ворожа ціль потрапляє в чергу на перехоплення (кінцева подія-повідомлення «Ціль у черзі»), інші треки залишаються під спостереженням. Модель реалізує функціональні вимоги FR-02, FR-04, FR-05 та FR-06 [4].

### *2.1.2. Процес призначення розрахунку та санкціонування пуску*

Другий процес (рис. 4.2) запускається повідомленням «Ціль у черзі», яке формує попередній процес. IoT-ядро автоматично підбирає розрахунки-кандидати та перевіряє деконфлікцію з планами польотів дружньої авіації і зонами заборони ураження. Офіцер бойового управління виконує користувацьке завдання «Обрати розрахунок», після чого оперативний черговий виконує завдання «Санкціонувати пуск».

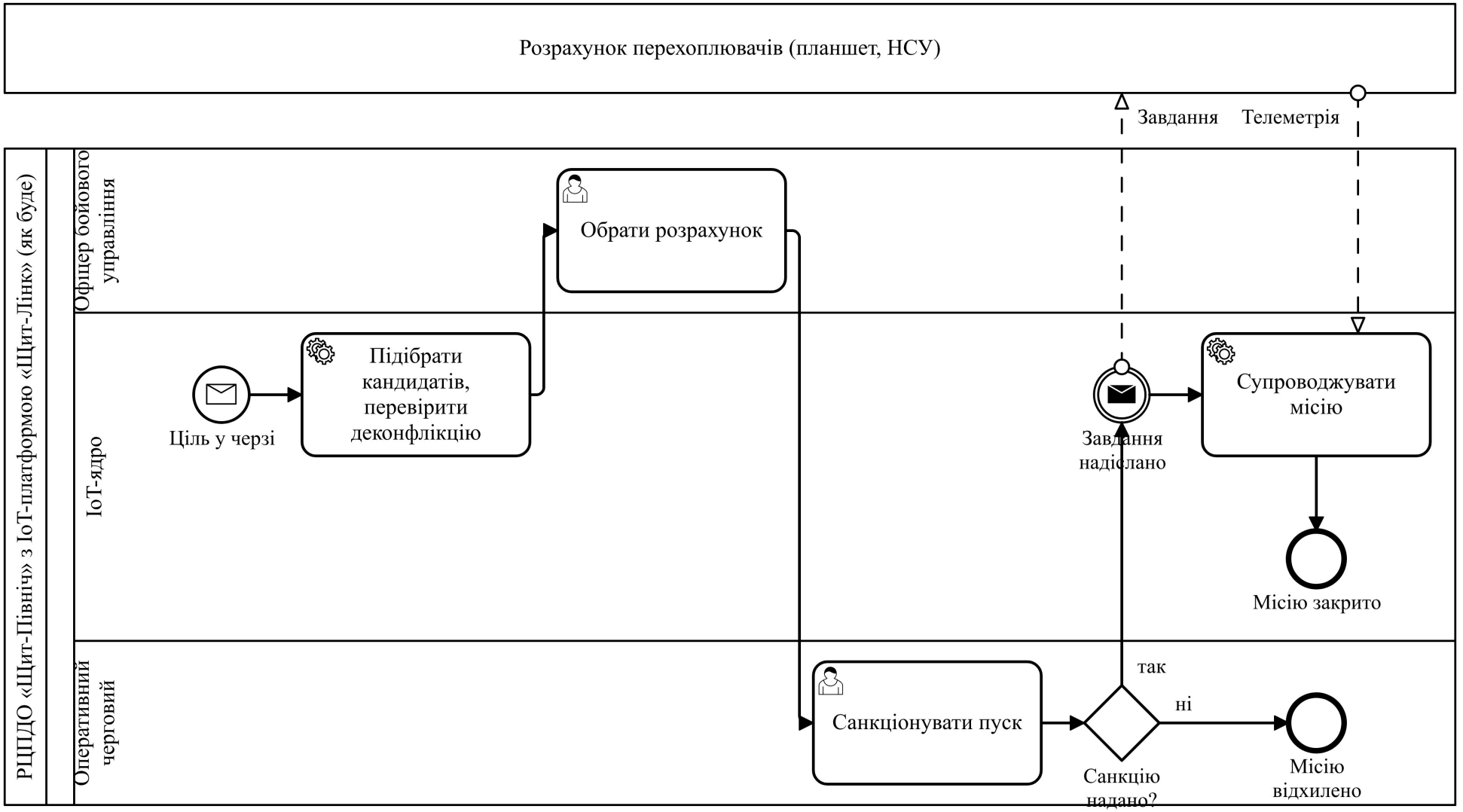



Рис. 4.2 — BPMN-модель процесу призначення розрахунку та санкціонування пуску

Шлюз «Санкцію надано?» розділяє потік: за відмови процес завершується подією «Місію відхилено», а після санкції проміжна подія-повідомлення «Завдання надіслано» передає бойове завдання до згорнутого пулу «Розрахунок перехоплювачів». Від розрахунку надходить телеметрія, яку використовує сервісне завдання «Супроводжувати місію»; процес завершується подією «Місію закрито». Розподіл дій між доріжками відображає принцип «людина в контурі»: система готує рішення, а обирає розрахунок і санкціонує пуск персонал (вимоги FR-08–FR-12, NFR-16 [4]).

## 2.2. Функціональна модель системи в нотації IDEF0

IDEF0 — методологія функціонального моделювання, в основі якої лежить поняття блоку, що відображає функцію; ліва сторона блоку означає вхід, права — вихід, верхня — управління, нижня — механізм [9]. Модель будується за принципами функціональної декомпозиції, обмеження складності (від двох до шести блоків на діаграмі) та контексту: спочатку будується контекстна діаграма з одним блоком, що фіксує межі системи [9]. Діаграми верхнього рівня називають батьківськими, а діаграми декомпозиції — дочірніми [10]. На відміну від моделі організації, тут модель побудовано з точки зору архітектора системи: механізмами є вузли самої системи.

### *2.2.1. Контекстна діаграма*

Контекстна діаграма (рис. 4.3) містить один блок A-0 «Забезпечити обмін телеметрією та координацію перехоплення». Входами системи є дані сенсорів і телеметрія перехоплювачів, а також рішення персоналу КП (класифікація, вибір розрахунку, санкції). Управління задають правила застосування, плани польотів дружньої авіації та вимоги захисту інформації. Виходи — повітряна обстановка, бойові завдання розрахункам, звіти та сповіщення. Механізмами є три групи вузлів архітектури ПР2 [5]: периферійні шлюзи, IoT-ядро та застосунки. У текстовому блоці зафіксовано мету моделювання та точку зору.

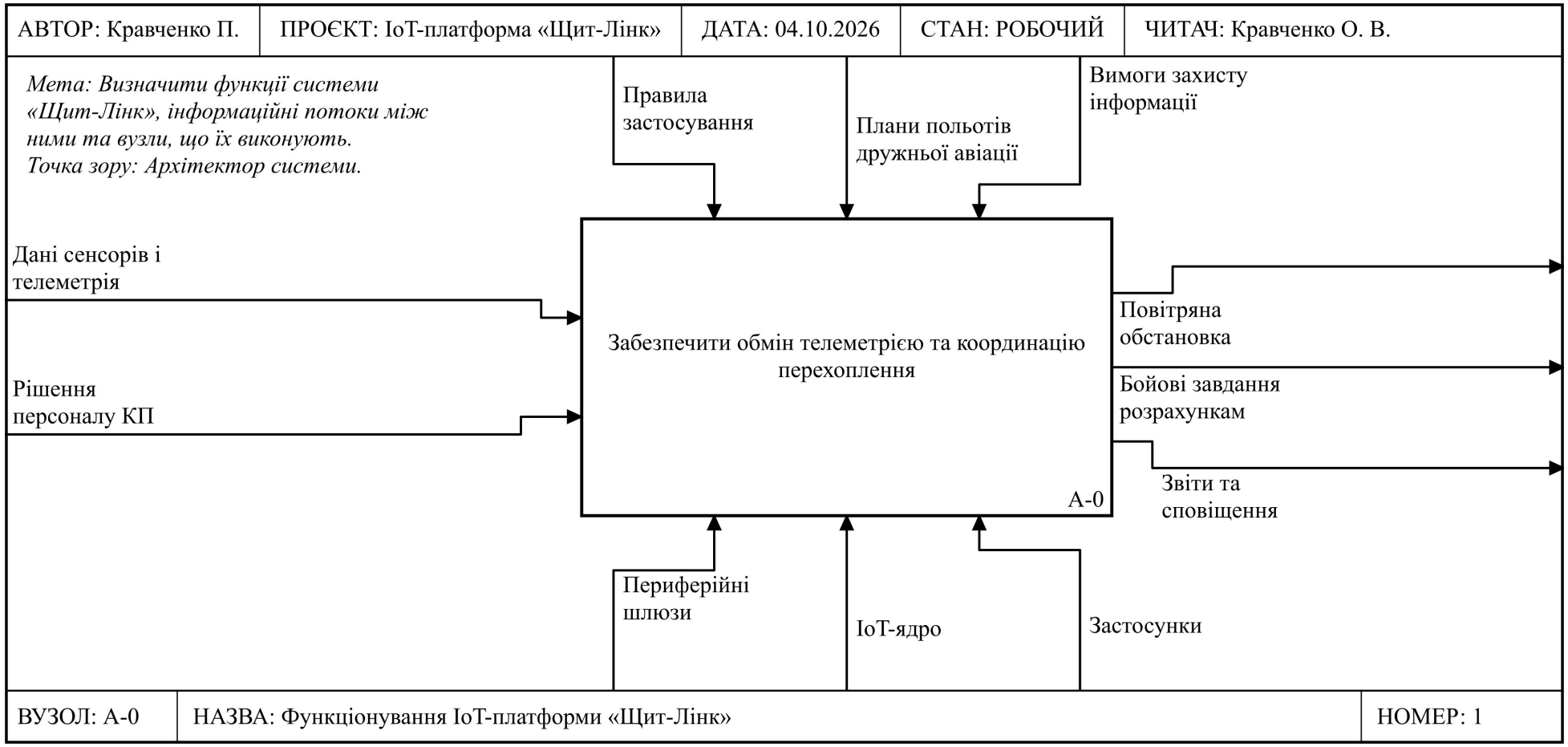



Рис. 4.3 — Контекстна діаграма IDEF0 системи «Щит-Лінк»

Розподіл межових стрілок за сторонами блоку визначає межі відповідальності системи. Стрілки входу відображають те, що система перетворює: потік повідомлень від сенсорів і перехоплювачів надходить без участі людини, тоді як рішення персоналу КП вводяться через автоматизовані робочі місця (АРМ). Стрілки управління не змінюються під час виконання функції, а лише обмежують її: правила застосування задають умови, за яких ціль може бути призначена на перехоплення, плани польотів дружньої авіації — простір, у якому призначення заборонене, вимоги захисту інформації — порядок автентифікації джерел і доступу користувачів. Стрілки виходу описують результати, що споживаються поза межами системи: повітряну обстановку переглядає персонал КП, бойові завдання отримують розрахунки перехоплювачів, звіти й сповіщення надходять до командування, техніків і логістів.

Рамка діаграми містить службові поля, прийняті в нотації IDEF0: у верхній частині — автора, назву проекту, дату, стан розроблення («робочий») і читача, тобто особу, що рецензує модель; у нижній частині — вузол A-0, назву діаграми та її порядковий номер. Нумерація вузла A-0 свідчить, що діаграма є контекстною, тобто розташованою над діаграмою верхнього рівня A0.

### *2.2.2. Діаграма декомпозиції першого рівня*

Декомпозиція A0 (рис. 4.4) розкриває чотири функції системи, розташовані за ступенями відповідно до порядку їх виконання:

– **A1 «Зібрати й нормалізувати дані»** отримує дані сенсорів і телеметрію (I1) під управлінням вимог захисту інформації (C3) та виконується периферійними шлюзами (M1); виходи — нормалізовані події для A2 і телеметрія місій для A3;

– **A2 «Сформувати повітряну обстановку»** під управлінням правил застосування (C1) формує повітряну обстановку (O1) та чергу цілей для A3; механізм — IoT-ядро (M2);

– **A3 «Координувати місії перехоплення»** отримує цілі в черзі, телеметрію місій і рішення персоналу КП (I2), враховує правила застосування та плани польотів (C1, C2), видає бойові завдання розрахункам (O2) та закриті місії для A4; виконується IoT-ядром і застосунками (M2, M3);

– **A4 «Облікувати ресурси та звітувати»** на основі закритих місій під управлінням вимог захисту інформації формує звіти та сповіщення (O3).

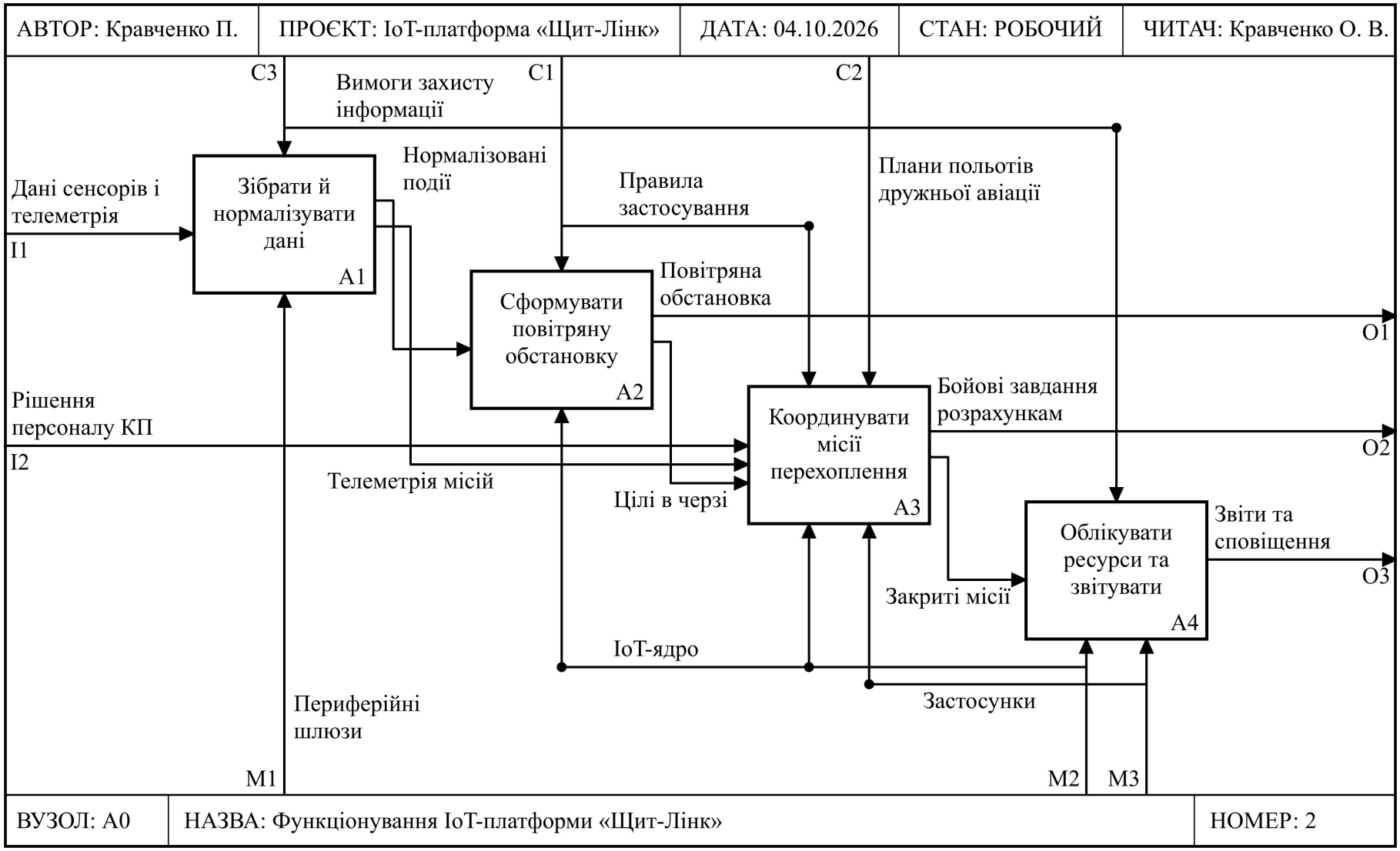



Рис. 4.4 — Діаграма декомпозиції A0 системи «Щит-Лінк»

Стрілки управління C1 і C3 та механізм M2 розгалужуються до кількох блоків (точки розгалуження). Діаграма показує, що периферійний шлюз відповідає лише за прийом і нормалізацію даних, а вся логіка обстановки та місій зосереджена в ядрі, що відповідає архітектурі ПР2 [5]. Кожна межова стрілка діаграми A0 має відповідник на контекстній діаграмі (коди I, C, O, M), що забезпечує узгодженість моделі.

## 2.3. Концептуальна модель системи в нотації UML

Візуальне моделювання в UML виконується як поступовий перехід від концептуальної моделі до логічної, а потім до фізичної [2]. Концептуальна модель описує функціональне призначення системи (діаграма прецедентів), поведінку ключової сутності (діаграма станів) і словник предметної області (діаграма класів).

### *2.3.1. Діаграма прецедентів*

Діаграма прецедентів (варіантів використання) визначає функціональне призначення системи: що система повинна робити, не розкриваючи, як саме вона це робить [2]. Діаграма (рис. 4.5) побудована в практичній роботі № 1 [4] і є вихідною концептуальною моделлю системи. Прямокутник позначає межу системи «IoT-платформа «Щит-Лінк»: збір даних і бойове управління»; актори розташовані поза нею, прецеденти — всередині. Лінія асоціації між актором і прецедентом означає, що актор ініціює прецедент або бере участь у ньому.

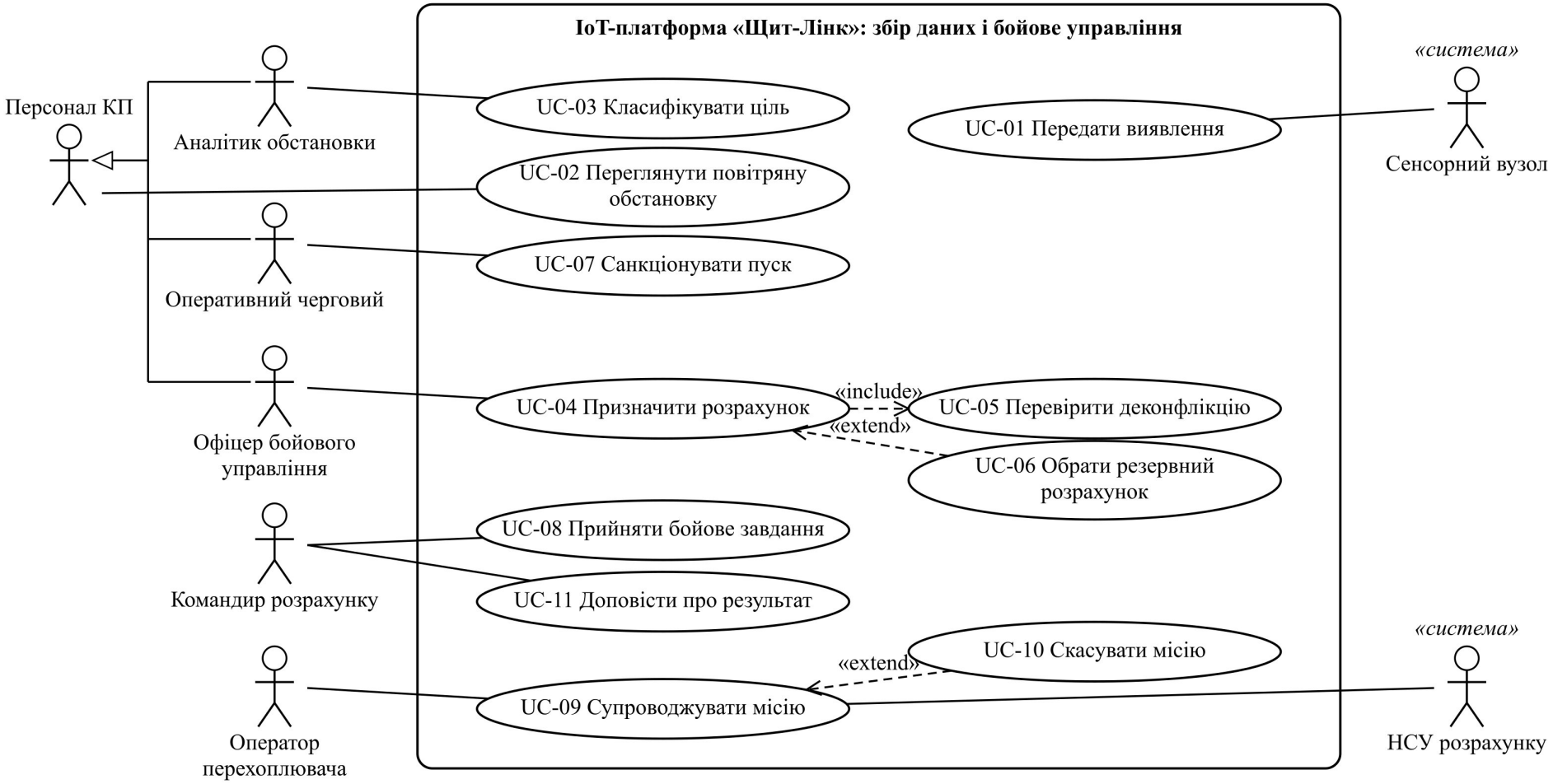



Рис. 4.5 — Діаграма прецедентів: збір даних і бойове управління (з ПР1)

На діаграмі визначено шість акторів-людей і двох акторів-систем:

– **Персонал КП** — абстрактний актор, що узагальнює ролі командного пункту; відношення узагальнення від аналітика обстановки, оперативного чергового та офіцера бойового управління означає, що кожна з цих ролей успадковує його прецедент «UC-02 Переглянути повітряну обстановку»;

– **Аналітик обстановки** веде повітряні треки та ініціює прецедент «UC-03 Класифікувати ціль»: підтверджує або змінює тип і належність цілі, запропоновані класифікатором;

– **Оперативний черговий** ініціює прецедент «UC-07 Санкціонувати пуск» — без цього прецедента жодне бойове завдання не може бути надіслане розрахунку;

– **Офіцер бойового управління** ініціює прецедент «UC-04 Призначити розрахунок», обираючи розрахунок для цілі з черги на перехоплення;

– **Командир розрахунку** бере участь у прецедентах «UC-08 Прийняти бойове завдання» та «UC-11 Доповісти про результат»;

– **Оператор перехоплювача** бере участь у прецеденті «UC-09 Супроводжувати місію» і підтверджує ціль на фазі наведення;

– **Сенсорний вузол** (актор-система) ініціює прецедент «UC-01 Передати виявлення» за протоколами MQTT або ASTERIX без участі людини;

– **НСУ розрахунку** (наземна станція управління, актор-система) передає телеметрію перехоплювача в межах прецедента «UC-09 Супроводжувати місію».

Між прецедентами встановлено три залежності. Відношення включення («include») від «UC-04 Призначити розрахунок» до «UC-05 Перевірити деконфлікцію» означає, що перевірка відсутності конфлікту з дружньою авіацією та зонами заборони ураження виконується за кожного призначення без винятків. Відношення розширення («extend») від «UC-06 Обрати резервний розрахунок» до «UC-04» описує альтернативний потік: резервний розрахунок обирається лише тоді, коли основний не підтвердив завдання протягом 15 с. Друге відношення розширення — від «UC-10 Скасувати місію»

до «UC-09 Супроводжувати місію» — застосовується за командою скасування під час виконання місії.

Прецеденти діаграми утворюють наскрізний ланцюжок «виявлення — класифікація — призначення — санкціонування — прийняття завдання — супровід — доповідь», який далі деталізують діаграми станів, діяльності, послідовності та кооперації. Специфікації основних прецедентів (короткий опис, основний і альтернативні потоки, передумови та постумови) наведено в ПР1 [4]; прецеденти забезпечення (облік техніки й запасів, звітність, адміністрування) винесено на окрему діаграму ПР1 і в цьому розділі не деталізуються.

### *2.3.2. Діаграма станів*

Діаграма станів моделює зміни стану прецедента як реакцію на зовнішні події [2]. Для «Щит-Лінк» найскладнішу поведінку має сутність «Місія» (рис. 4.6). Місія створюється в стані «Запропонована»; за відмови оперативного чергового переходить у стан «Відхилена», після санкції — у стан «Санкціонована». Якщо розрахунок не підтвердив завдання протягом 15 с, місія стає «Перепризначеною» і повертається до стану «Запропонована» з новим кандидатом. Після підтвердження місія переходить до станів «Прийнята» та «Виконується» (подія «пуск»), звідки завершується станом «Закрита» за доповіддю про результат або «Скасована» за командою скасування. Стани «Відхилена», «Скасована» й «Закрита» є кінцевими.

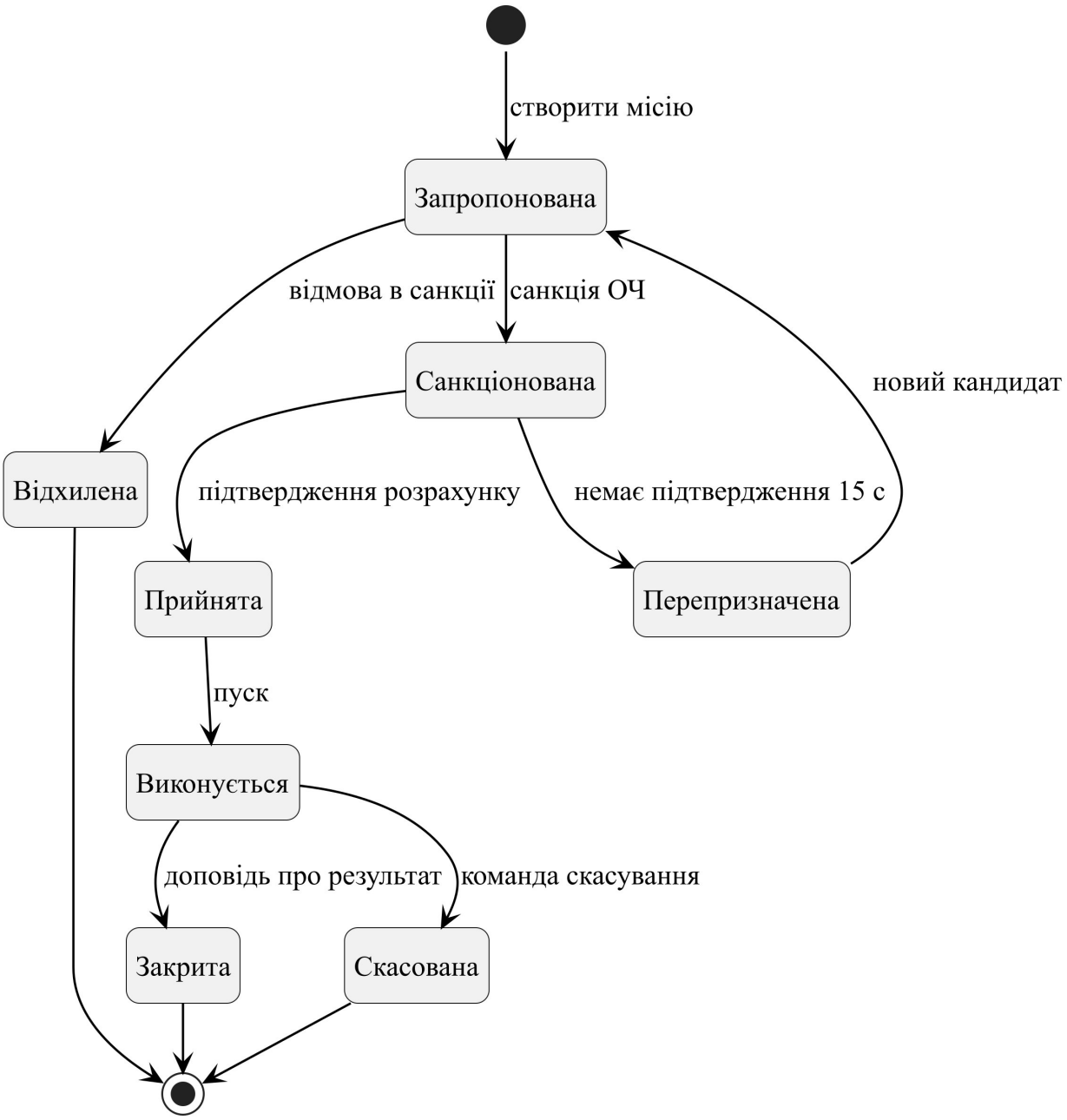



Рис. 4.6 — Діаграма станів сутності «Місія»

Кожен перехід позначено подією, що його спричиняє, а ініціатор події відповідає актору діаграми прецедентів: створення місії — офіцеру бойового управління (UC-04), санкція або відмова — оперативному черговому (UC-07), підтвердження — командиру розрахунку (UC-08), доповідь про результат — прецеденту UC-11, скасування — прецеденту UC-10. Перехід «немає підтвердження 15 с» ініціює сама система за таймером; він реалізує розширення «UC-06 Обрати резервний розрахунок». Цикл «Санкціонована — Перепризначена — Запропонована» гарантує, що зміна розрахунку потребує

нової санкції оперативного чергового, тобто призначення не обходить людину. Перелік станів діаграми безпосередньо використано як множину допустимих значень стовпця state таблиці mission фізичної моделі (п. 2.6).

### *2.3.3. Діаграма класів концептуального рівня*

На концептуальному рівні класи використовуються для складання словника системи: виділяються класи, їх внутрішня структура та взаємозв’язки [2]. Діаграма (рис. 4.7) містить вісім класів:

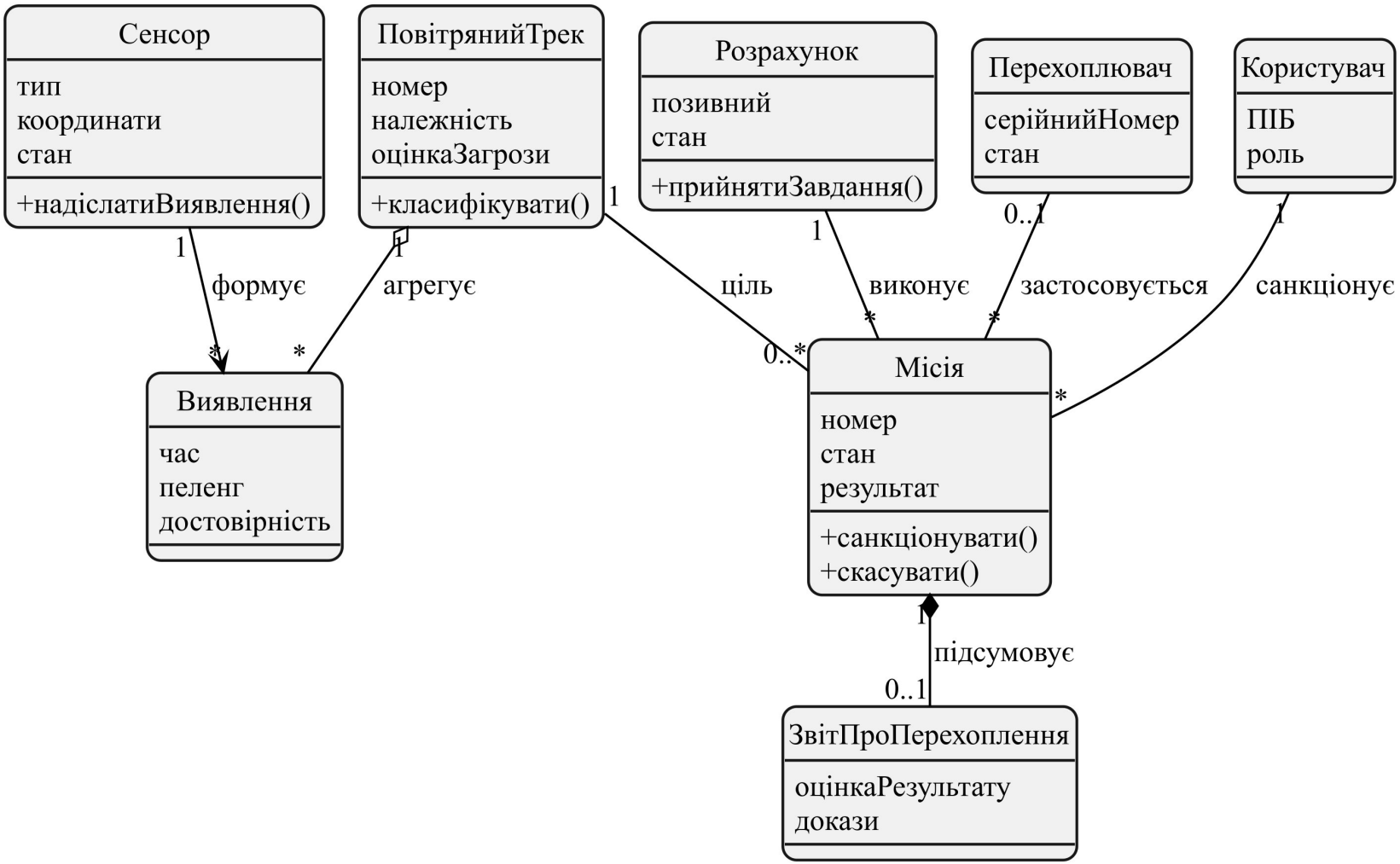



Рис. 4.7 — Діаграма класів концептуального рівня

– **Сенсор** — тип, координати, стан; операція надіслатиВиявлення() формує об’єкти класу «Виявлення» (асоціація 1 — *);

– **Виявлення** — час, пеленг, достовірність; об’єкти агрегуються в «Повітряний трек» (агрегація);

– **ПовітряннийТрек** — номер, належність, оцінка загрози; операція класифікувати() фіксує рішення аналітика; один трек може бути ціллю кількох місій (0..*);

– **Місія** — номер, стан, результат; операції санкціонувати() і скасувати() реалізують переходи діаграми станів (рис. 4.6);

– **Розрахунок** — позивний, стан; операція прийнятиЗавдання(); один розрахунок виконує багато місій;

– **Перехоплювач** — серійний номер, стан; у кожній місії застосовується не більше одного перехоплювача;

– **Користувач** — ПІБ, роль; оперативний черговий санкціонує місії;

– **ЗвітПроПерехоплення** — оцінка результату, докази; пов’язаний з місією композицією (0..1), тобто не існує без місії.

Між класами встановлено відношення трьох видів. Направлена асоціація «формує» від класу «Сенсор» до класу «Виявлення» з кратністю 1 — * означає, що кожне виявлення має рівно одне джерело, а один сенсор формує необмежену кількість виявлень; напрямок вказує, що виявлення не знає про подальшу обробку. Агрегація (незафарбований ромб) між класами «ПовітряннийТрек» і «Виявлення» відображає відношення «ціле — частина», за якого частини існують незалежно: виявлення зберігаються для аналізу навіть після закриття треку. Композиція (зафарбований ромб) між класами «Місія» і «ЗвітПроПерехоплення» є сильнішою формою агрегації: звіт створюється лише для конкретної місії та видаляється разом з нею.

Клас «Місія» є центральним: він пов’язаний асоціаціями з п’ятьма іншими класами. Кратність 0..* на боці місії в асоціації «ціль» з класом «ПовітряннийТрек» допускає трек без місій (ціль під спостереженням) і повторне призначення на ту саму ціль після промаху. Кратність 0..1 на боці

класу «Перехоплювач» означає, що до пуску перехоплювач місії не визначено, а після пуску застосовується рівно один. Асоціація «санкціонує» з класом «Користувач» фіксує персональну відповідальність посадової особи за кожен пуск. Атрибути класів наведено без типів, оскільки на концептуальному рівні важливий склад даних, а не спосіб їх зберігання; типи визначаються у фізичній моделі (рис. 4.11).

## 2.4. Логічна модель системи

Логічна модель уточнює концептуальну у вигляді, зручному для подальшої реалізації, але без прив’язки до конкретної платформи [2]. Для ключового сценарію «Призначення розрахунку» побудовано діаграми діяльності, послідовності та кооперації.

### *2.4.1. Діаграма діяльності*

Діаграма діяльності деталізує алгоритмічну логіку операцій: перехід до наступної дії відбувається лише після завершення попередньої [2]. Діаграма (рис. 4.8) розділена на чотири доріжки: офіцер бойового управління (ОБУ), платформа, оперативний черговий (ОЧ) і розрахунок. ОБУ обирає ціль; платформа підбирає кандидатів і перевіряє деконфлікцію — у разі невдачі процес завершується блокуванням призначення. Далі ОБУ обирає розрахунок, платформа створює місію, а ОЧ ухвалює рішення про санкцію. Після санкції платформа надсилає завдання, і розрахунок має прийняти його протягом 15 с; інакше платформа пропонує резервний розрахунок. Сценарій завершується супроводом місії.

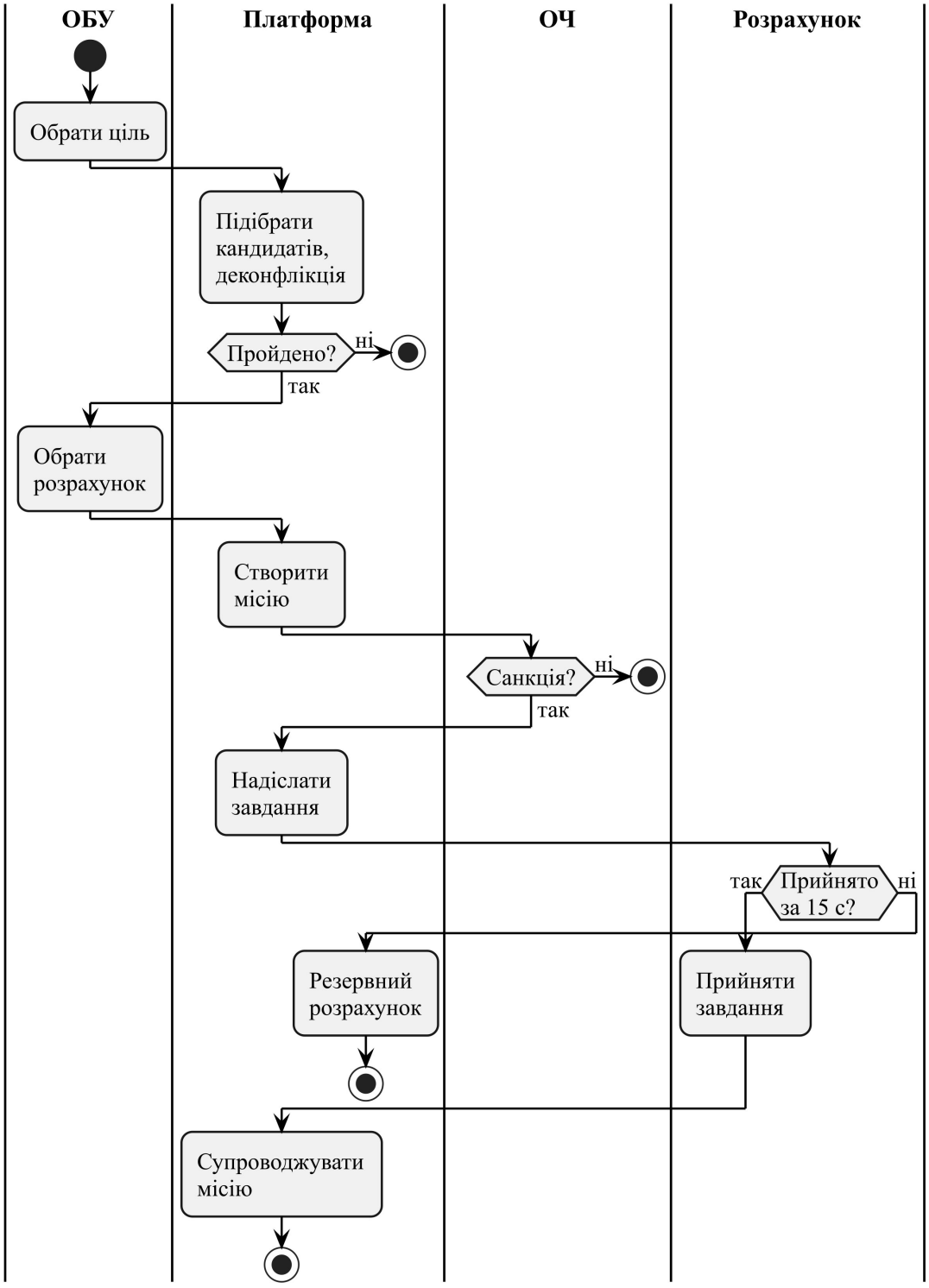



Рис. 4.8 — Діаграма діяльності сценарію «Призначення розрахунку»

Діаграма побудована з таких елементів: початкового вузла (зафарбоване коло), дій (прямокутники із заокругленими кутами), вузлів розгалуження (ромби з умовою та гілками «так» / «ні») і кінцевих вузлів (коло з обідком). Доріжки (swimlanes) показують, хто відповідає за кожну дію, тому перехід

потоку управління між доріжками означає передачу відповідальності між учасниками. Послідовність дій така:

1) Дія «Обрати ціль» у доріжці ОБУ — офіцер обирає повітряний трек із черги на перехоплення.

2) Дія «Підібрати кандидатів, деконфлікція» у доріжці платформи — сервіс призначення формує перелік розрахунків, здатних досягти цілі, і перевіряє відсутність конфлікту з дружньою авіацією.

3) Розгалуження «Пройдено?»: за наявності конфлікту виконується кінцевий вузол — призначення блокується і місія не створюється.

4) Дія «Обрати розрахунок» у доріжці ОБУ — офіцер обирає одного з запропонованих кандидатів.

5) Дія «Створити місію» у доріжці платформи — місія отримує стан «Запропонована».

6) Розгалуження «Санкція?» у доріжці ОЧ: за відмови сценарій завершується, місія переходить у стан «Відхилена».

7) Дія «Надіслати завдання» у доріжці платформи — бойове завдання доставляється на планшет розрахунку.

8) Розгалуження «Прийнято за 15 с?» у доріжці розрахунку: гілка «так» веде до дії «Прийняти завдання», гілка «ні» — до дії платформи «Резервний розрахунок», після якої поточний сценарій завершується і запускається повторно з новим кандидатом.

9) Дія «Супроводжувати місію» у доріжці платформи та кінцевий вузол завершують основний потік.

Діаграма містить чотири кінцеві вузли, кожен з яких відповідає окремому результату сценарію. Така структура дає змогу безпосередньо отримати з діаграми тестові сценарії: по одному на кожен шлях від початкового до кінцевого вузла.

### *2.4.2. Діаграма послідовності*

Діаграма послідовності відображає динаміку взаємодії об’єктів у часі; на ній показано лише об’єкти, які безпосередньо беруть участь у взаємодії [2].

Ініціатором послідовності (рис. 4.9) є актор «ОБУ», описаний на діаграмі прецедентів. Під час взаємодії виконуються такі операції:

– **обратиЦіль()** — ОБУ обирає ціль на АРМ КП;

– **кандидати()** — АРМ запитує в сервісу «Призначення і місії» перелік розрахунків-кандидатів;

– **деконфлікція()** — сервіс синхронно перевіряє в геосервісі відсутність конфлікту з дружньою авіацією та зонами заборони ураження й отримує відповідь «дозволено»;

– **призначити(розрахунок)**, **створитиМісію()** — ОБУ призначає розрахунок, сервіс створює місію та публікує подію «місію запропоновано» в шину подій;

– **санкціонувати()** — ОЧ, отримавши місію через шину подій, надає санкцію; сервіс публікує подію «місію санкціоновано», і шина доставляє бойове завдання на планшет;

– **прийнято()** — розрахунок підтверджує прийняття; подія «завдання прийнято» повертається до сервісу, який оновлює стан місії на АРМ.

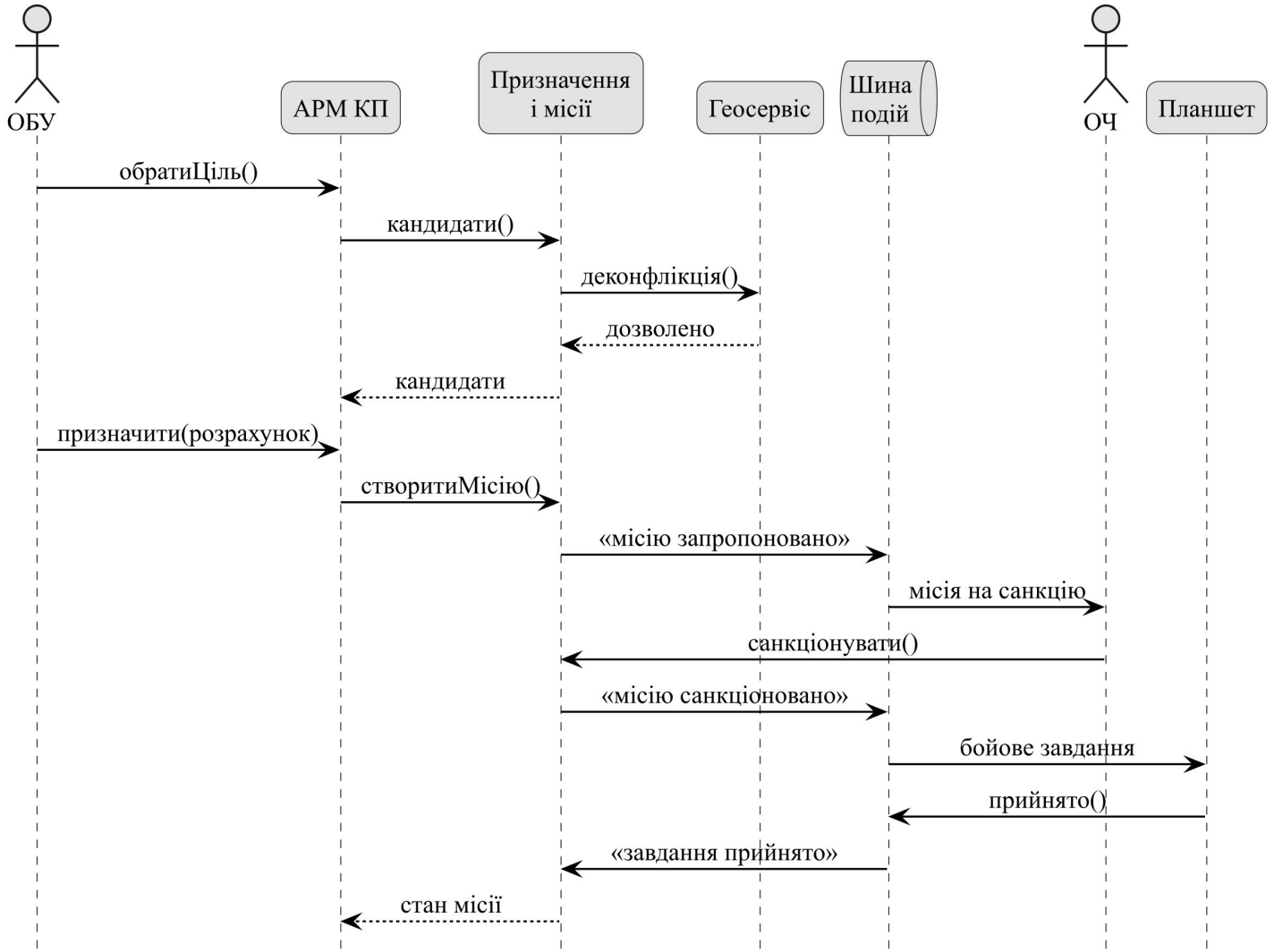



Рис. 4.9 — Діаграма послідовності сценарію «Призначення розрахунку»

Учасники взаємодії розташовані горизонтально в порядку першої участі: актор «ОБУ», об’єкти «АРМ КП» (інтерфейс користувача), «Призначення і місії» (сервіс, що керує сценарієм) і «Геосервіс», черга «Шина подій», актор «ОЧ» та «Планшет» розрахунку. Вертикальні пунктирні лінії (лінії життя) відображають існування об’єктів протягом сценарію; час на діаграмі плине згори донизу. Суцільні стрілки із зафарбованим наконечником позначають виклики операцій, пунктирні — повернення результату (відповіді «дозволено», «кандидати», «стан місії»). Повідомлення, взяті в лапки («місію запропоновано», «місію санкціоновано», «завдання прийнято»), є подіями, які публікуються в шину і доставляються підписникам.

Синхронні виклики з очікуванням відповіді використовуються лише всередині ядра — між АРМ, сервісом призначення та геосервісом, — оскільки результат деконфлікції потрібен до продовження сценарію. Взаємодія з ОЧ і планшетом відбувається асинхронно через шину подій: сервіс призначення не очікує санкції, а реагує на подію, коли вона надходить. Такий розподіл відповідає подієво-орієнтованій архітектурі ПР2 [5] і дає змогу планшету отримати завдання навіть після тимчасової втрати зв’язку, оскільки подія зберігається в шині до доставки.

### *2.4.3. Діаграма кооперації*

Діаграма кооперації описує взаємодію об’єктів, абстрагуючись від часу; порядок повідомлень задано нумерацією [2]. Центральним об’єктом (рис. 4.10) обрано анонімний об’єкт класу «Місія», який приймає та генерує такі повідомлення:

– **1: створити(трек, розрахунок)** — ініціатор ОБУ; вхідні параметри — ідентифікатори треку й розрахунку; результат — місія в стані «Запропонована»;

– **2: отриматиОцінкуЗагрози()** і **3: перевіритиДеконфлікцію()** — генерує «Місія»; власники операцій — об’єкти «ПовітряннийТрек» і «Геосервіс»; результат — пріоритет цілі та дозвіл на призначення;

– **4: санкціонувати()** — ініціатор ОЧ; результат — стан «Санкціонована»;

– **5: надіслатиЗавдання()** — генерує «Місія»; власник — «Планшет», який передає **6: прийнятиЗавдання()** об’єкту «Розрахунок»;

– **7: підтвердити()** — розрахунок підтверджує прийняття; результат — стан «Прийнята».

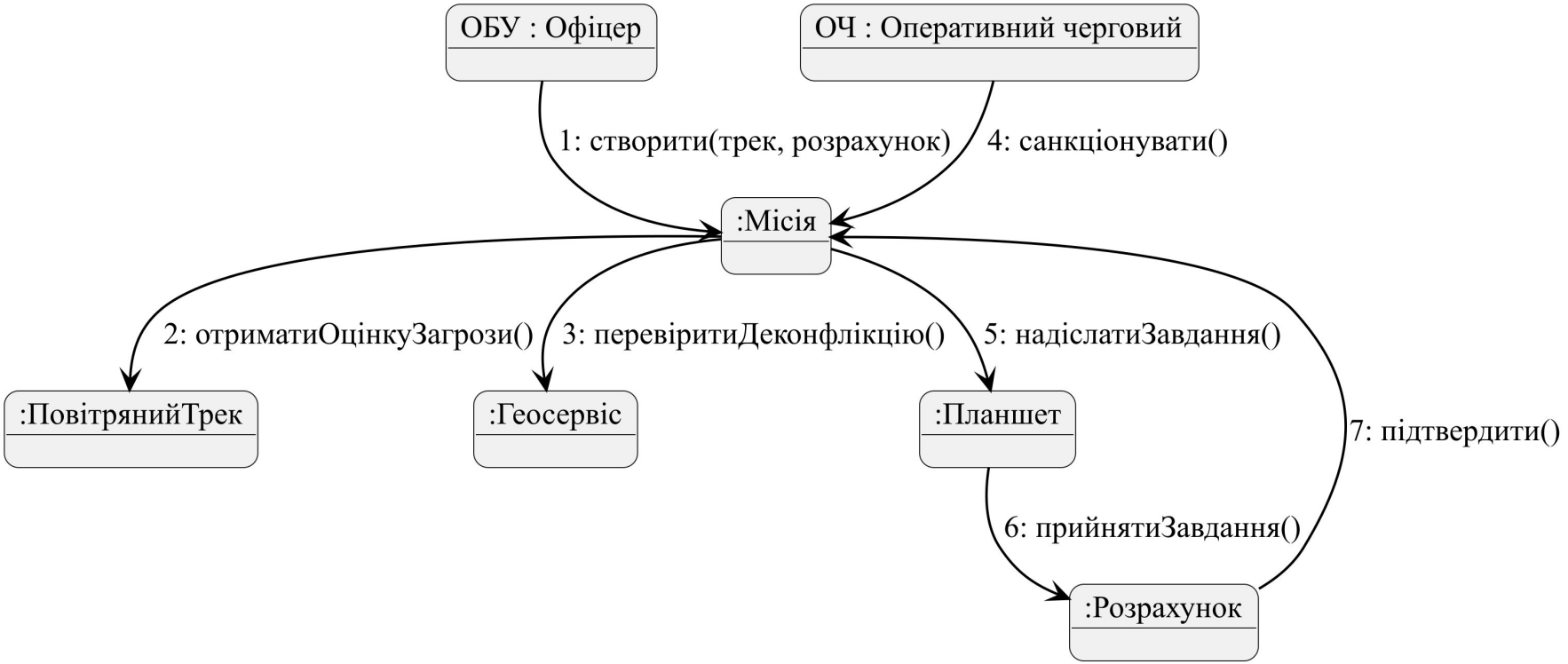



Рис. 4.10 — Діаграма кооперації з центральним об’єктом «Місія»

Об’єкти діаграми позначено в нотації «ім’я : клас»: іменовані об’єкти «ОБУ : Офіцер» та «ОЧ : Оперативний черговий» відповідають акторам діаграми прецедентів, анонімні об’єкти (з двокрапкою перед іменем класу) — екземплярам класів концептуальної моделі та сервісам ядра. Лінії між об’єктами є зв’язками (links), тобто екземплярами асоціацій діаграми класів; уздовж кожного зв’язку показано номер, напрямок і назву повідомлення.

На відміну від діаграми послідовності, яка робить наочним порядок повідомлень у часі, діаграма кооперації показує структуру зв’язків: з неї видно, що об’єкт «Місія» має шість зв’язків і є єдиною точкою узгодження сценарію, тоді як «Розрахунок» взаємодіє з місією лише опосередковано — через планшет. Повідомлення 2 і 3 генерує сама місія, тобто перевірка загрози та деконфлікції інкапсульована в класі «Місія», а не покладена на користувача. Обидві діаграми взаємодії описують той самий сценарій і можуть бути перетворені одна в одну.

### 2.5. Фізична модель системи

Фізична модель визначає способи розміщення даних і компонентний склад системи з точки зору реалізації на конкретній технічній базі [2].

#### *2.5.1. Діаграма класів фізичного рівня*

Фізична діаграма класів описує структуру даних у термінах платформи реалізації — СКБД PostgreSQL (рис. 4.11). Кожен клас відповідає таблиці (стереотип «table»), атрибути мають типи СКБД (UUID, VARCHAR, ENUM, DECIMAL, TIMESTAMPTZ, BOOL), первинні ключі позначено PK, зовнішні — FK. Центральна таблиця mission містить зовнішні ключі на трек (track_id), розрахунок (crew_id), перехоплювач (interceptor_id, може бути порожнім до пуску) і користувача, що санкціонував пуск (approved_by). Таблиця engagement_report пов’язана з mission відношенням 1 : 0..1. Повна схема бази даних з 11 таблиць наведена в ПР1 [4].

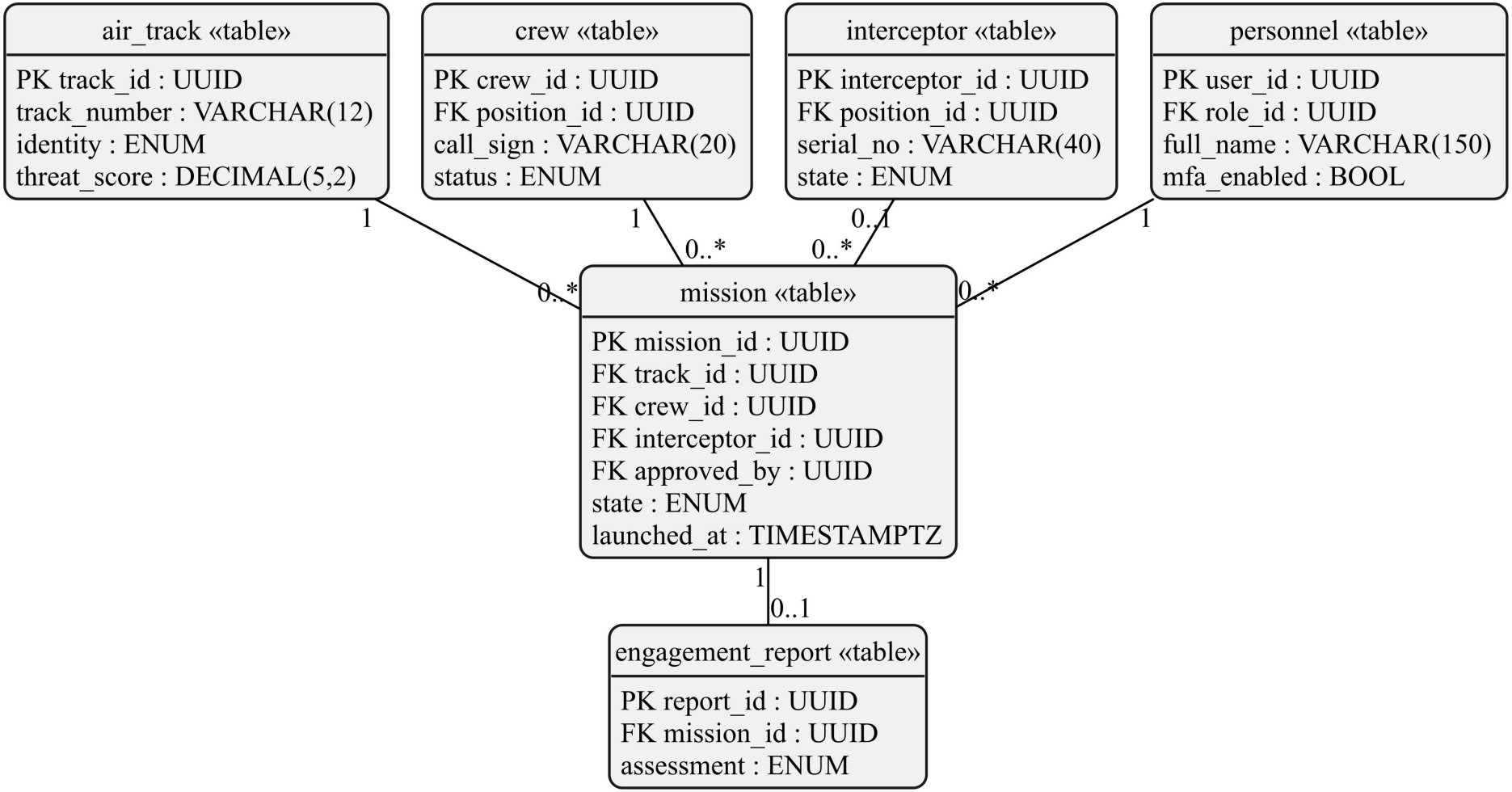



Рис. 4.11 — Діаграма класів фізичного рівня (фрагмент схеми бази даних)

Перехід від концептуальної моделі до фізичної виконано за такими правилами:

– кожен клас предметної області, екземпляри якого зберігаються, перетворено на таблицю з англомовним ідентифікатором (клас «Місія» — таблиця mission, «ПовітряннийТрек» — air_track, «Користувач» — personnel тощо);

– кожна таблиця отримала сурогатний первинний ключ типу UUID, що генерується без звернення до центрального лічильника і тому придатний для розподіленої системи з кількома ЦОД;

– асоціації з кратністю «1 — багато» перетворено на зовнішні ключі на боці «багато»: таблиця mission містить track_id, crew_id, interceptor_id та approved_by;

– кратність 0..1 перетворено на стовпець, що допускає порожнє значення (interceptor_id до пуску), а композицію «Місія — Звіт» — на зовнішній ключ mission_id у таблиці engagement_report з обмеженням унікальності, що забезпечує відношення 1 : 0..1;

– атрибути з обмеженою множиною значень (належність треку, стан місії, оцінка результату) отримали тип ENUM; перелік станів місії збігається зі станами діаграми на рис. 4.6;

– моменти часу зберігаються в типі TIMESTAMPTZ з часовим поясом, оскільки події надходять від вузлів, розташованих у різних місцях, а журнал має бути однозначно впорядкований.

Зовнішні ключі position_id і role_id посилаються на таблиці «Позиція» та «Роль», які не наведені у фрагменті, щоб діаграма залишалася читабельною.

### *2.5.2. Діаграма компонентів*

Діаграма компонентів моделює статичний вигляд системи з точки зору реалізації [2]. Компоненти (рис. 4.12) згруповано в три пакети. Пакет «Периферійний шлюз» містить адаптери протоколів, локальний брокер MQTT і компонент «Буфер і міст до ядра». Пакет «IoT-ядро» містить шину подій, сервіси реального часу, бізнес-сервіси, API-шлюз і сховища PostgreSQL та

TimescaleDB. Пакет «Застосунки» містить веб-АРМ і мобільний застосунок, які звертаються до ядра лише через API-шлюз. Пунктирна залежність від мобільного застосунку до локального брокера показує роботу планшета через шлюз позиції без зв’язку з ядром.

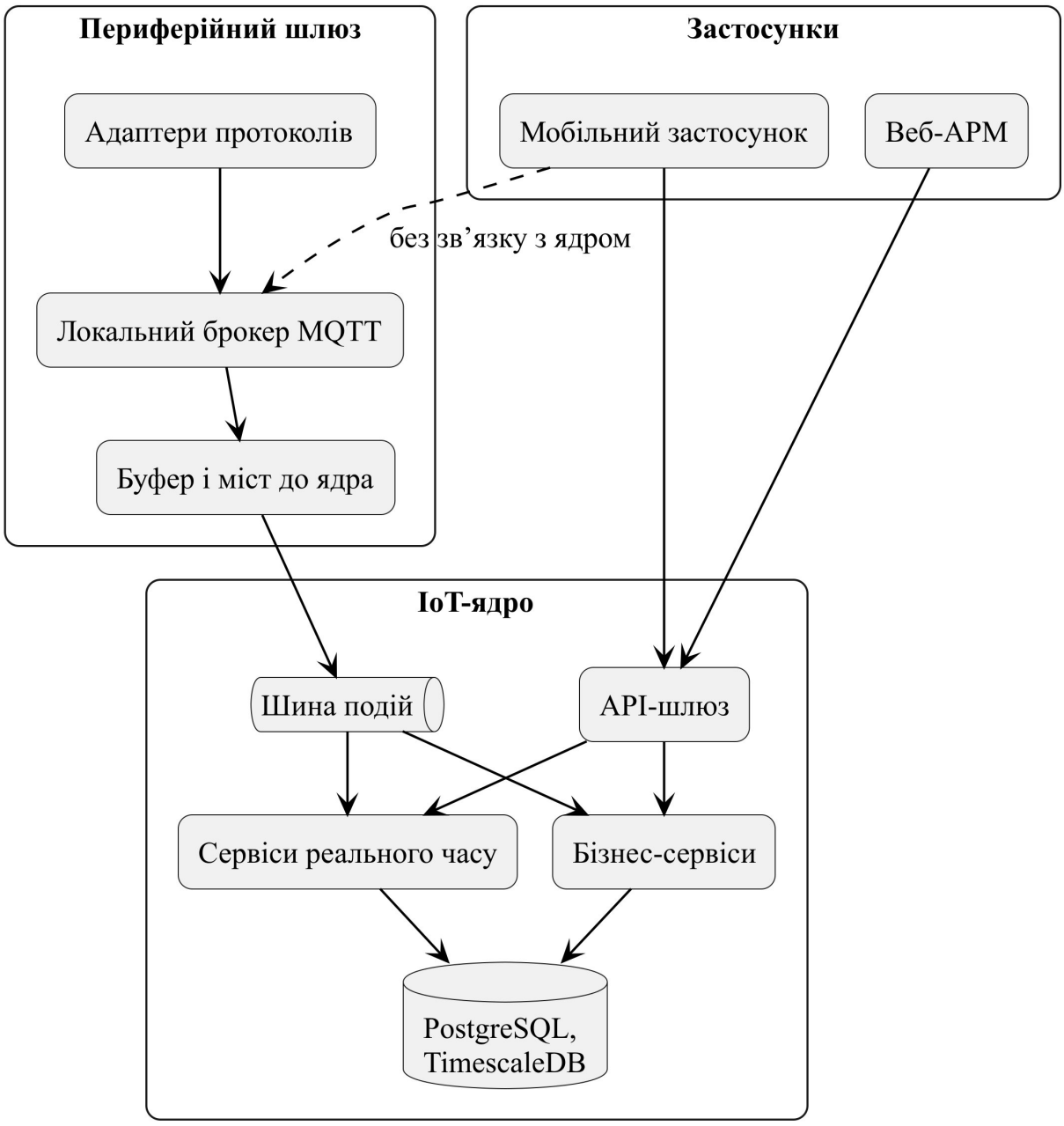



Рис. 4.12 — Діаграма компонентів системи «Щит-Лінк»

Стрілки залежностей на діаграмі показують напрямок звернення одного компонента до іншого і разом утворюють такі маршрути даних:

– **висхідний потік телеметрії:** адаптери протоколів приймають повідомлення сенсорів і перетворюють їх у єдиний формат, локальний брокер

MQTT розподіляє їх на позиції, а компонент «Буфер і міст до ядра» зберігає повідомлення під час втрати зв’язку та передає їх до шини подій ядра;

– **обробка в ядрі:** шина подій доставляє події двом групам підписників — сервісам реального часу (злиття треків, оцінка загрози) та бізнес-сервісам (призначення, місії, облік ресурсів); обидві групи зберігають дані в реляційному сховищі PostgreSQL і часовому сховищі TimescaleDB;

– **доступ користувачів:** веб-АРМ і мобільний застосунок звертаються лише до API-шлюзу, який перевіряє права доступу й маршрутизує запити до сервісів; прямого звернення застосунків до бази даних немає.

Така структура реалізує принцип слабкої зв’язаності: компоненти периферійного шлюзу не залежать від компонентів ядра, крім єдиної точки входу — шини подій, а застосунки залежать лише від інтерфейсу API-шлюзу. Заміна або масштабування окремого сервісу ядра не потребує змін у шлюзах і застосунках.

### *2.5.3. Діаграма розгортання*

Діаграма розгортання (рис. 4.13) подає конфігурацію та топологію розподіленої системи:

– вузол «Позиція» (9 екземплярів) містить периферійний шлюз на платформі NVIDIA Jetson Orin Nano, обраній у ПР3 [6], і планшет розрахунку на Android, з’єднаний зі шлюзом через Wi-Fi із шифруванням TLS;

– сенсори сектора передають дані на шлюз за протоколами MQTT і ASTERIX;

– шлюз з’єднаний з кластером ядра в основному ЦОД через VPN із взаємною автентифікацією (mTLS);

– кластер ядра працює з базою даних і реплікує дані до резервного ЦОД;

– АРМ командного пункту звертаються до ядра через HTTPS.

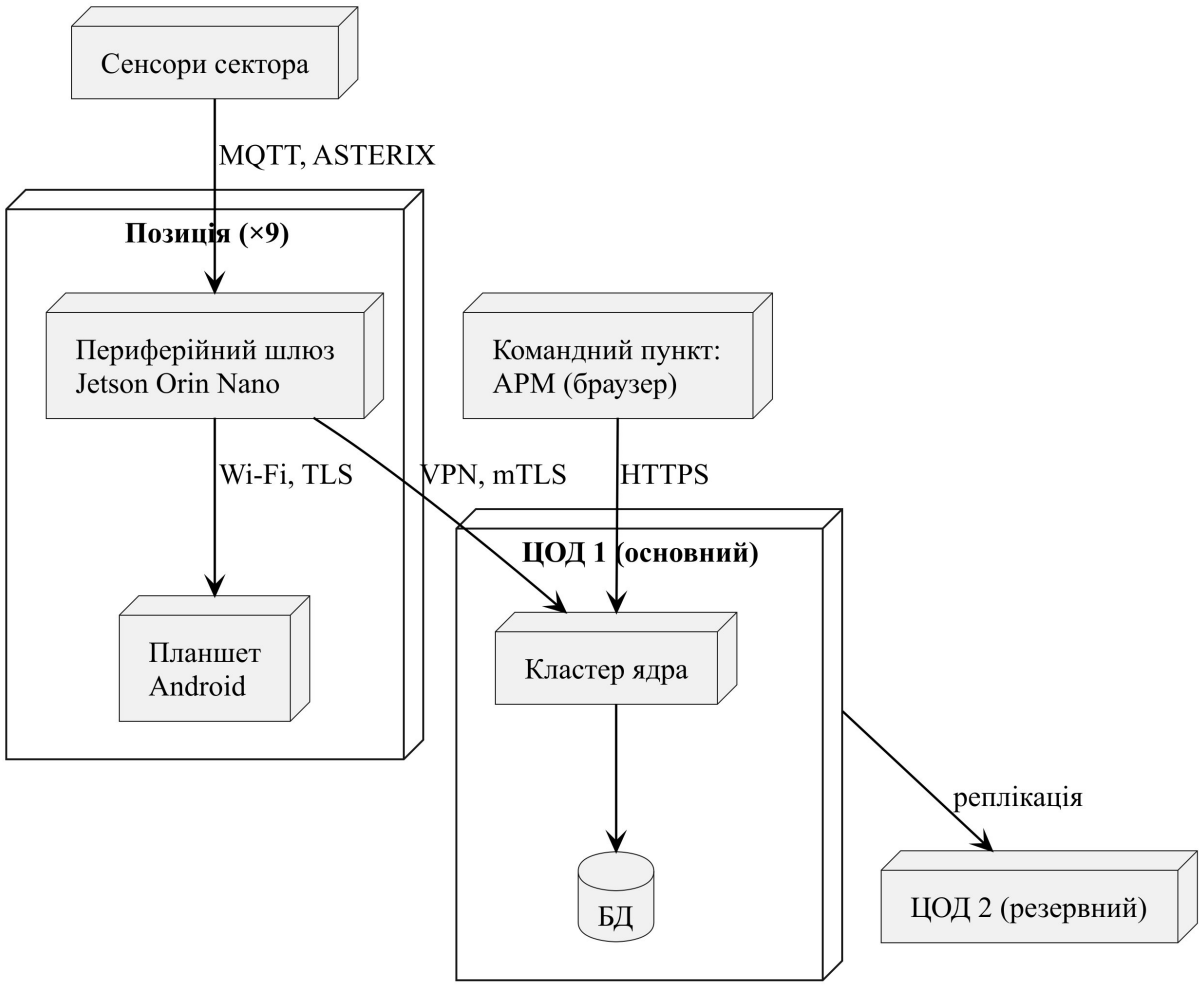



Рис. 4.13 — Діаграма розгортання системи «Щит-Лінк»

Вузли діаграми розгортання є фізичними обчислювальними ресурсами, на яких виконуються компоненти (рис. 4.12): пакет «Периферійний шлюз» розгортається на вузлі «Периферійний шлюз Jetson Orin Nano», пакет «IoT-ядро» — на вузлі «Кластер ядра» з базою даних, мобільний застосунок — на планшеті Android, а веб-АРМ — у браузері робочих місць командного пункту. Вкладеність вузлів означає фізичне розміщення: шлюз і планшет перебувають на тій самій позиції та з’єднані локальною мережею Wi-Fi, тому планшет розрахунку продовжує отримувати дані сенсорів сектора через локальний брокер шлюзу навіть за тимчасової відсутності каналу до ЦОД.

Підписи на з’єднаннях між вузлами задають протоколи та засоби захисту каналів. Усі канали, що виходять за межі позиції або ЦОД, захищені: між шлюзом і ядром — віртуальна приватна мережа з взаємною автентифікацією

за сертифікатами, між АРМ і ядром — протокол HTTPS. Реплікація даних до резервного ЦОД підтримує вимоги до доступності ядра (NFR-05, не менше 99,95 % на місяць) і відновлення (NFR-06, RPO не більше 5 с, RTO не більше 60 с), а розміщення шлюзу на позиції — вимогу автономної роботи шлюзу протягом не менше 4 год без зв’язку з ядром (FR-03, NFR-06) [4].

Взаємозв’язок вузлів і модулів, показаний на діаграмах компонентів і розгортання, узгоджується з IDEF0-моделлю (механізми M1–M3 на рис. 4.4) і з BPMN-моделями (доріжки «Периферійний шлюз» та «IoT-ядро» на рис. 4.1, 4.2).

### 2.6. Реалізація моделей у CASE-засобах

CASE-засоби (Computer-Aided Software Engineering) автоматизують аналіз і проектування програмних систем: побудову діаграм, перевірку їх узгодженості, підготовку проектної документації та генерацію програмного коду з моделей [2]. До засобів проектування та моделювання належать UML-інструменти (StarUML, Enterprise Architect) та інструменти для створення архітектури (Draw.io, Lucidchart); окрему категорію становлять системи керування версіями (Git), що відстежують зміни та зберігають історію [11].

Для моделей розділу застосовано підхід «модель як код»: кожна діаграма зберігається у вигляді текстового опису моделі у стандартному або відкритому форматі, а зображення формуються автоматично. Перевагою підходу є можливість зберігати моделі в репозиторії Git разом з іншими артефактами проекту, порівнювати їх версії рядок за рядком і повторно формувати всі рисунки після зміни моделі без ручного перемальовування. Склад використаних засобів наведено в табл. 4.2.

Таблиця 4.2 — Засоби моделювання, використані в розділі

| Нотація | Засіб | Формат моделі | Можливості |
|---|---|---|---|
| BPMN | bpmn-js — засіб перегляду та редагування моделей BPMN 2.0 [12] | XML-файл *.bpmn за специфікацією BPMN 2.0.2 [13] з розділом графічного подання (координати елементів) | Модель відкривається для перегляду та редагування в будь-якому засобі, сумісному з BPMN 2.0 |
| IDEF0 | Генератор діаграм IDEF0 на мові Python в оформленні AllFusion Process Modeler | Перелік блоків і стрілок з кодами ICOM (вхід, управління, вихід, механізм) | Автоматична маршрутизація стрілок, розгалуження, рамка з полями вузла, назви й номера |
| UML | PlantUML — засіб із відкритим кодом, що будує UML-діаграми з текстового опису [14] | Текстові файли *.puml | Діаграми станів, класів, діяльності, послідовності, кооперації, компонентів і розгортання; формати SVG і PNG |
| Схема БД | Генератор SQL DDL (пряме проектування) | Діаграма класів фізичного рівня *.puml | Скрипт створення таблиць, ключів і переліків для СКБД PostgreSQL |

Послідовність обробки моделей показано на рис. 4.14. Вихідні моделі трьох нотацій зберігаються в репозиторії Git. Засоби bpmn-js, генератор IDEF0 та PlantUML формують з них векторні та растрові зображення, після чого кожне зображення проходить автоматичний контроль читабельності: розмір шрифту на сторінці звіту має бути не менше 10 пт, а роздільна здатність — не менше 300 dpi. Рисунки, що пройшли контроль, включаються до проектної документації. Діаграма класів фізичного рівня, крім того, передається генератору SQL DDL.

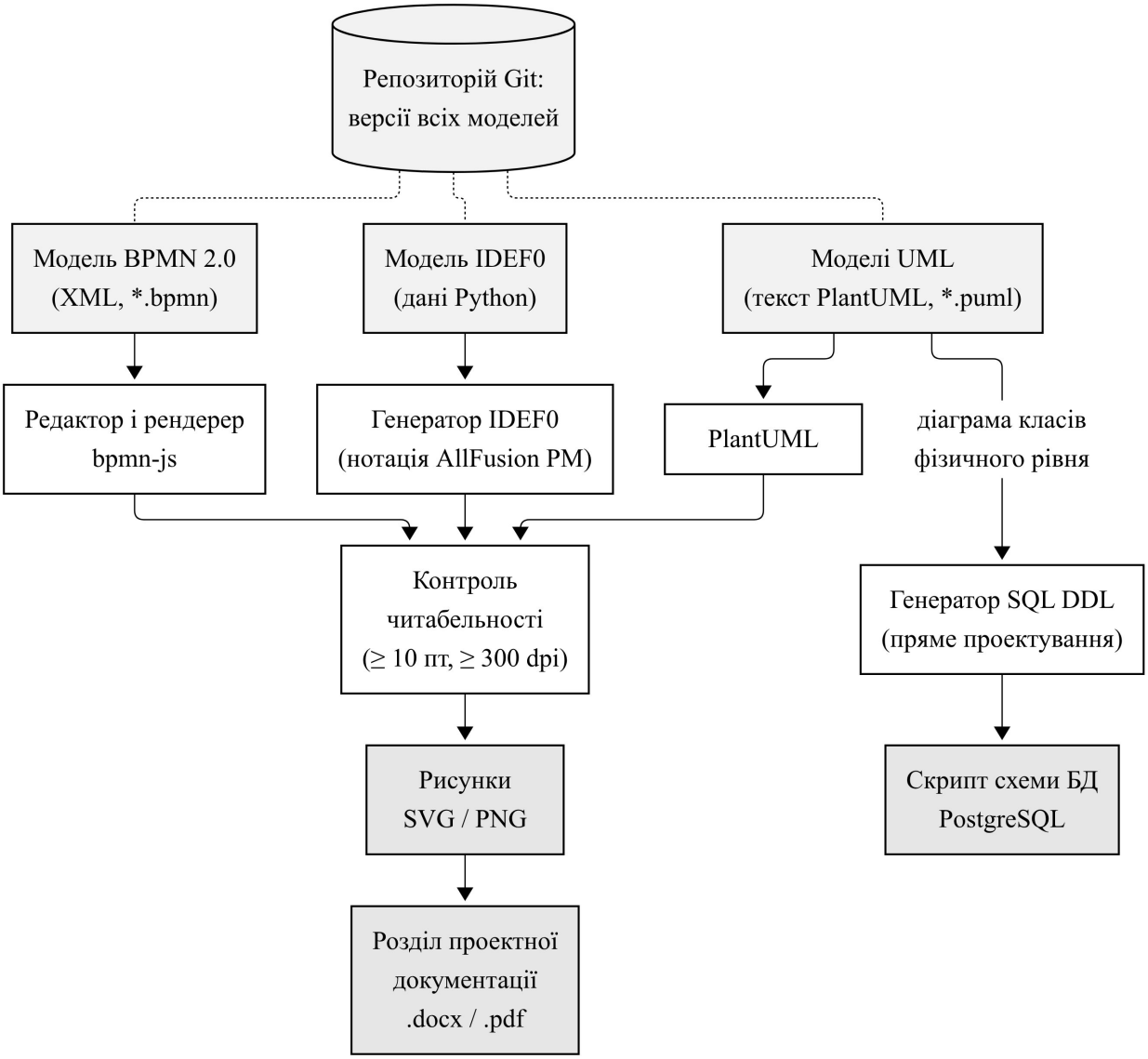



Рис. 4.14 — Схема обробки моделей проектування

Найважливішою властивістю CASE-засобу є можливість генерувати програмний код з побудованої моделі [2]. Для системи «Щит-Лінк» пряме проектування (forward engineering) виконано для схеми бази даних: генератор зчитує діаграму класів фізичного рівня (рис. 4.11) і діаграму станів (рис. 4.6) та формує скрипт мовою SQL для СКБД PostgreSQL за такими правилами:

– клас зі стереотипом «table» перетворюється на оператор CREATE TABLE, атрибут з позначкою PK — на первинний ключ із генерацією значення UUID;

– атрибут з позначкою FK перетворюється на зовнішній ключ (REFERENCES); таблиця, на яку він посилається, визначається за збігом імені

стовпця з первинним ключем або, якщо імена відрізняються (approved_by), за асоціацією діаграми;

– кратність 0..1 на боці таблиці, на яку посилається ключ, дозволяє порожнє значення, а кратність 0..1 на боці таблиці з ключем додає обмеження унікальності (UNIQUE), що реалізує відношення 1 : 0..1;

– тип ENUM перетворюється на оператор CREATE TYPE; значення типу mission_state беруться безпосередньо зі станів діаграми станів сутності «Місія»;

– таблиці впорядковуються так, щоб таблиця, на яку посилається зовнішній ключ, створювалася раніше.

Фрагмент згенерованого скрипту наведено на рис. 4.15; повний скрипт містить п’ять типів-переліків і шість таблиць. Скрипт перевірено виконанням у СКБД PostgreSQL 18: усі оператори виконано без помилок, а структура таблиці mission відповідає діаграмі — чотири зовнішні ключі, з яких interceptor_id і approved_by допускають порожнє значення до пуску та санкції відповідно.

```
-- Схема БД IoT-платформи «Щит-Лінк» (фрагмент).
-- Згенеровано з діаграми класів фізичного рівня;
-- не редагувати вручну.

CREATE TYPE mission_state AS ENUM (
    'запропонована',
    'відхилена',
    'санкціонована',
    'перепризначена',
    'прийнята',
    'виконується',
    'скасована',
    'закрита'
);

CREATE TABLE mission (
    mission_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    track_id UUID NOT NULL
        REFERENCES air_track (track_id),
    crew_id UUID NOT NULL REFERENCES crew (crew_id),
    interceptor_id UUID
        REFERENCES interceptor (interceptor_id),
    approved_by UUID REFERENCES personnel (user_id),
    state mission_state NOT NULL,
    launched_at TIMESTAMPTZ
);

CREATE TABLE engagement_report (
    report_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    mission_id UUID NOT NULL
        REFERENCES mission (mission_id) UNIQUE,
    assessment engagement_report_assessment NOT NULL
);
```

Рис. 4.15 — Фрагмент SQL-скрипту, згенерованого з діаграми класів фізичного рівня

Повторна генерація після зміни діаграми оновлює скрипт автоматично, тому фізична модель і схема бази даних не розходяться. Зв’язок між діаграмою станів і типом mission_state забезпечує, що новий стан місії, доданий до моделі поведінки, одразу стає допустимим значенням у базі даних. Таким чином, результати моделювання перетворюються не лише на проектну документацію, а й на програмний артефакт, придатний для використання на етапі реалізації.

# ВИСНОВКИ

У практичній роботі виконано проектування IoT-платформи «Щит-Лінк» засобами трьох нотацій.

1) У нотації BPMN побудовано моделі «як буде» двох ключових процесів області застосування — формування повітряної обстановки та призначення розрахунку з санкціонуванням пуску; моделі показують розподіл дій між периферійним шлюзом, IoT-ядром і персоналом та обмін повідомленнями із сенсорною мережею й розрахунками.

2) У нотації IDEF0 побудовано контекстну діаграму та декомпозицію першого рівня з чотирма функціями системи; механізмами моделі є вузли системи (периферійні шлюзи, IoT-ядро, застосунки), що пов’язує функціональну модель з архітектурою.

3) У нотації UML побудовано концептуальну (прецедентів, станів, класів), логічну (діяльності, послідовності, кооперації) та фізичну (класів, компонентів, розгортання) моделі; вони описують призначення системи, життєвий цикл місії, взаємодію об’єктів у сценарії призначення розрахунку, схему даних, модулі та вузли.

4) Моделі реалізовано в CASE-засобах за підходом «модель як код» (bpmn-js, PlantUML, генератор IDEF0) під керуванням версій Git; з діаграми класів фізичного рівня автоматично згенеровано SQL-скрипт схеми бази даних для PostgreSQL, працездатність якого перевірено виконанням у СКБД.

5) Моделі узгоджені між собою та з результатами попередніх практичних робіт: вимогами ПР1, архітектурою ПР2 і обладнанням ПР3.

Отримані моделі є основою для реалізації програмного продукту відповідно до обраної комбінованої моделі життєвого циклу.

# СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Практична робота 4. Проектування системи : завдання з дисципліни «Технології створення програмних продуктів». Київ : КНУ імені Тараса Шевченка, 2026.

2. Розробка UML-моделі інформаційної системи «Овочева база» : приклад опису діаграм курсової роботи. Київ, 2020. 18 с.

3. Неделько Д., Дружинін В. Система управління реверсним рухом на Дніпровському узвозі у місті Києві з використанням IoT-технологій. Інформаційні системи та технології: результати і перспективи : матеріали 2-ї Міжнар. наук.-практ. конф. (IST 2025), м. Київ, 5 берез. 2025 р. Київ : ФІТ КНУТШ, 2025. С. 105–107.

4. Кравченко П. Збір та документування вимог для програмного продукту : звіт з практичної роботи № 1 з дисципліни «Технології створення програмних продуктів». Київ : КНУ імені Тараса Шевченка, 2026. 42 с.

5. Кравченко П. Архітектура IoT екосистеми : звіт з практичної роботи № 2 з дисципліни «Технології створення програмних продуктів». Київ : КНУ імені Тараса Шевченка, 2026. 24 с.

6. Кравченко П. Опис існуючих програмних та апаратних рішень : звіт з практичної роботи № 3 з дисципліни «Технології створення програмних продуктів». Київ : КНУ імені Тараса Шевченка, 2026. 25 с.

7. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. Київ : ДП «УкрНДНЦ», 2016. 26 с.

8. Технологія RUP : лекція 8 з дисципліни «Технології створення програмних продуктів» / викл. О. В. Кравченко. Київ : КНУ імені Тараса Шевченка, 2026. 46 слайдів.

9. Дегтярьова Л. М., Гроза П. М., Сомов С. В. Навчальний посібник з дисципліни «Технології розробки програмного забезпечення». Полтава : ПолтНТУ, 2017. 218 с.

10. Карпенко М. Ю., Манакова Н. О., Гавриленко І. О. Технології створення програмних продуктів та інформаційних систем : навч. посібник. Харків : ХНУМГ ім. О. М. Бекетова, 2017. 93 с.

11. Засоби та інструменти розробки ПЗ : лекція 7 з дисципліни «Технології створення програмних продуктів» / викл. О. В. Кравченко. Київ : КНУ імені Тараса Шевченка, 2026. 17 слайдів.

12. bpmn-js: BPMN 2.0 viewer and editor / bpmn.io. URL: https://bpmn.io/toolkit/bpmn-js/ (дата звернення: 11.10.2026).

13. Business Process Model and Notation (BPMN). Version 2.0.2 / Object Management Group. 2014. URL: https://www.omg.org/spec/BPMN/2.0.2/ (дата звернення: 11.10.2026).

14. PlantUML: open-source tool for UML diagrams. URL: https://plantuml.com/ (дата звернення: 11.10.2026).